

Getting Started with App

Hectron

August 13, 2026

Contents

1	Description	2
2	Run Application	2
3	Figures	2
4	Load Temperature Train Data into DB	5
4.1	Loading data into database	5
5	View Database Table	6
6	Tensorflow Model Export	7

1 Description

Document informs the user how to start the application residing in the `nba_app` directory of repository `ReactApps`. Applications were developed on MacOS *Sonoma*. The following programming languages were used throughout fullstack development.

- **FrontEnd**
Typescript - imperative
- **Server:**
Typescript - imperative
- **Backend:**
Postgres - declarative
- **Script:**
Bash - imperative

2 Run Application

- Open Docker. If Docker is not installed, please download at <https://www.docker.com/products/docker-desktop/>
- Start/Configure Postgres Database (Docker Container Instance)
 - Change directory to `nba_app/database`
 - run `run.sh`
- Start Monitor Service.
The highest priority service. All request are handled by the monitor service. The start monitor must be running for proper operation of the application.
 - Change directory to `nba_app`
 - run `server/main.sh` *Monitor service is now running*
- Open web browser; request application at address and port `http://127.0.0.1:50214/`

3 Figures

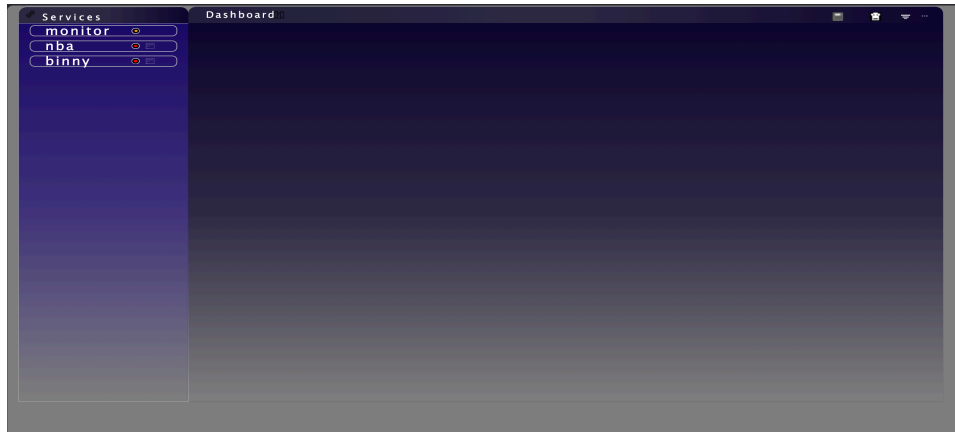


Figure 1: Home Page



Figure 2: GET NBA Service

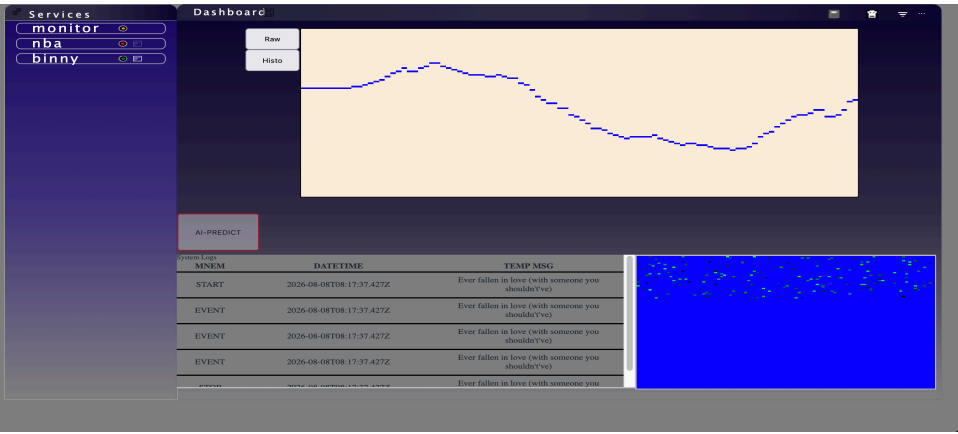


Figure 3: GET Binny Service - XY Plot

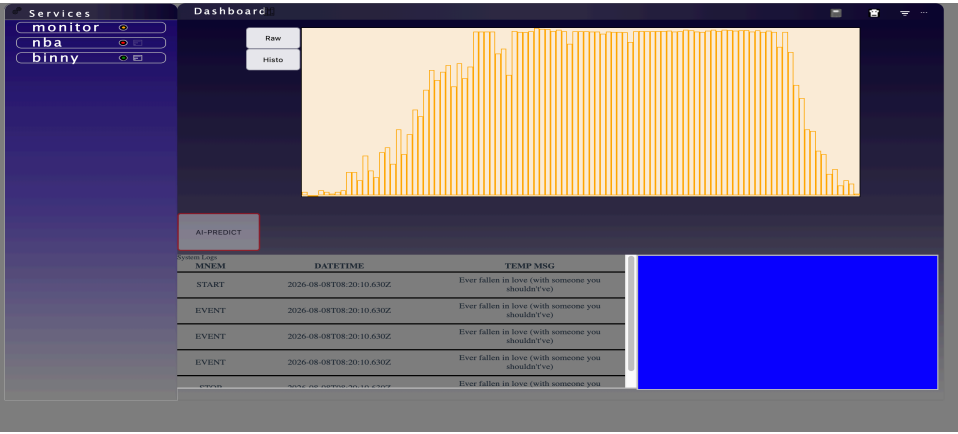


Figure 4: GET Binny Service - Histo

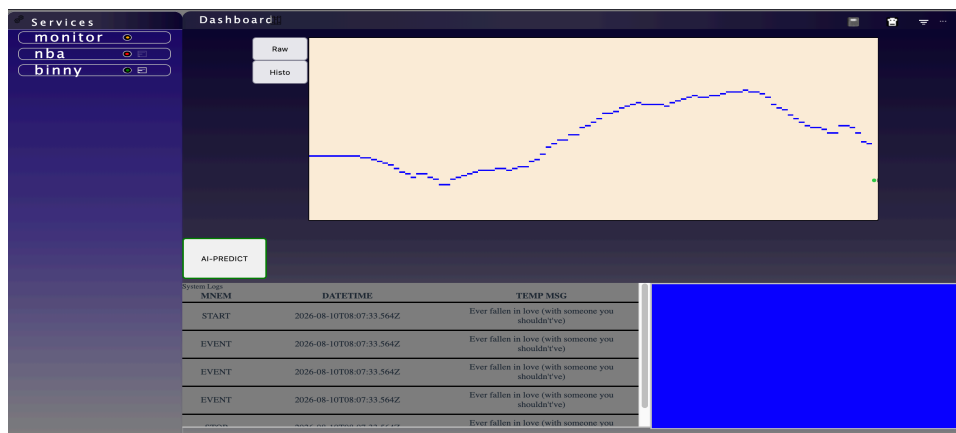


Figure 5: **GET** Model Prediction Service

4 Load Temperature Train Data into DB

We used temperature data from Kaggle <https://www.kaggle.com/datasets/bhanupratapbiswas/weather-data> to represent log data. The prediction model will be used predict potential future logs; this will be discussed in a future section.

4.1 Loading data into database

- Download WeatherData.csv aforementioned above and lowercase the filename.
- Move weatherdata.csv into nba_app\database\pgfiles directory
- Create sql file (e.g. index2.sql) with schema matching the features labeled in WeatherData.csv

```
CREATE TABLE binny.temp (  
    date_time text,  
    temp real ,  
    dew_point_temp real,  
    humidity decimal,  
    wind_speed decimal,  
    visibility decimal,  
    weather text  
);
```

- Create script copyweather.sh in nba_app\database\pgfiles
copyweather.sh

```
DB=sport_db  
USER=httron  
psql -d "${DB}" --username "${USER}" <<< "\copy binny.temp \  
FROM '/root/work/WeatherData.csv' DELIMITER ',' CSV HEADER"
```

- Add the following commands to nba_app\database\run.sh
run.sh

```
DB=sport_db  
USER=httron  
docker exec "${CONTAINER_ID}" psql -d "${DB}" --username "${USER}" -f /root/work/index2.sql  
  
docker cp ./pgfiles/copyweather.sh \  
<CONTAINER_ID>:/root/work/copyweather.sh  
  
docker cp ./pgfiles/weatherdata.csv \  
<CONTAINER_ID>:/root/work/weatherdata.csv  
  
stdout=$(docker exec "${CONTAINER_ID}" \  
./root/work/copyweather.sh & )  
echo $stdout
```

5 View Database Table

Let's view the weather table

- Access PostgreSQL container (i.e. username=htron)

```
docker exec -it nba-pg psql -d sport\db --username htron
```

- Verify binny table exist

```
sport\_db=> \dt binny.*
```

```
      List of tables
Schema | Name | Type | Owner
-----+-----+-----+-----
binny  | temp | table | htron
```

```
sport_db=> select * from binny.temp;
```

date_time	temp	dew_point_temp	humidity	wind_speed	visibility	kpa	weather
1/1/2012 0:00	-1.8	-3.9	86	4	8	101.24	Fog
1/1/2012 1:00	-1.8	-3.7	87	4	8	101.24	Fog
1/1/2012 2:00	-1.8	-3.4	89	7	4	101.26	Freezing Drizzle,Fog
1/1/2012 3:00	-1.5	-3.2	88	6	4	101.27	Freezing Drizzle,Fog
1/1/2012 4:00	-1.5	-3.3	88	7	4.8	101.23	Fog
1/1/2012 5:00	-1.4	-3.3	87	9	6.4	101.27	Fog
1/1/2012 6:00	-1.5	-3.1	89	7	6.4	101.29	Fog
1/1/2012 7:00	-1.4	-3.6	85	7	8	101.26	Fog
1/1/2012 8:00	-1.4	-3.6	85	9	8	101.23	Fog
1/1/2012 9:00	-1.3	-3.1	88	15	4	101.2	Fog
1/1/2012 10:00	-1	-2.3	91	9	1.2	101.15	Fog
1/1/2012 11:00	-0.5	-2.1	89	7	4	100.98	Fog
1/1/2012 12:00	-0.2	-2	88	9	4.8	100.79	Fog
1/1/2012 13:00	0.2	-1.7	87	13	4.8	100.58	Fog
1/1/2012 14:00	0.8	-1.1	87	20	4.8	100.31	Fog
1/1/2012 15:00	1.8	-0.4	85	22	6.4	100.07	Fog
1/1/2012 16:00	2.6	-0.2	82	13	12.9	99.93	Mostly Cloudy
1/1/2012 17:00	3	0	81	13	16.1	99.81	Cloudy
1/1/2012 18:00	3.8	1	82	15	12.9	99.74	Rain
1/1/2012 19:00	3.1	1.3	88	15	12.9	99.68	Rain
1/1/2012 20:00	3.2	1.3	87	19	25	99.5	Cloudy

6 Tensorflow Model Export

A tensorflow built sequence-to-vector network is exported into our Binny service. Our AI-Predict button infers the model. The following steps were used to create and load model.

- Create virtual environment using python 3.10
- Install package housing command-line utility tensorflowjs_converter :

```
pip install --upgrade protobuf
pip install --upgrade tensorflowjs
```

- Train dataset (i.e. temperature in this case) using your preferred network of choice.
- Networks

- DNN Stateless Model

```
s_model = tf.keras.models.Sequential()
s_model.add(keras.Input(shape=(N_SAMPLES,)))
s_model.add(keras.layers.Dense(N_SAMPLES, activation='selu'))
s_model.add(keras.layers.Dense(N_SAMPLES / 2, activation='selu'))
s_model.add(keras.layers.Dense(N_SAMPLES / 3 + 1, activation='selu'))
```

- DNN Recurrent Network

```
input_ = tf.keras.layers.Input(shape=(N_SAMPLES, 1), batch_size = 32, dtype = tf.float32, name= "layer_in")
x = keras.layers.GRU(1, return_sequences=True, unroll=True, activation="selu", name="layer_gru1")(input_)
x = keras.layers.GRU(1, unroll=True, activation="selu", name="layer_gru2")(x)
output_ = keras.layers.Dense(1, name="layer_dense_out")(x)
s_model = keras.Model(inputs=input_, outputs=output_)
```

- Convert python keras model to Tensorflow.js

```
tensorflowjs_converter --input_format=tf_saved_model --output_format=tfjs_graph_model $PWD/saved_model $PWD/web_model
```

- Export model *The exported model must use static shapes and not contain dynamic control flow operations(i.e. while loops in graph model).*
- Make server route to infer model
- Validate (Test event/inference)